

Tikki auf einer Seite

Was diese App ist, wie sie gebaut ist und wie man sie betreibt – als Prompt für Google NotebookLM. Stand 2. Oktober 2026, Repo **TikkiAL-boop/hermes-agent**, Zweig **tikki-app**.

Prompt an NotebookLM: Du bist mein Erklärer für mein Softwareprojekt **Tikki**. Nimm diese Seite als einzige Quelle. Erkläre mir auf Nachfrage in einfachem Deutsch, was Tikki kann, welche Teile es gibt, wie sie zusammenhängen und was noch offen ist. Bleibe bei dem, was hier steht; wenn etwas nicht drinsteht, sag das. Verwende die Begriffe genau so wie hier (Raum, Raumleiter, Übersicht, Briefing, Dauerauftrag, Tür, Verschmelzen, Gateway, Profil, Rolle).

1 • Kurz gesagt

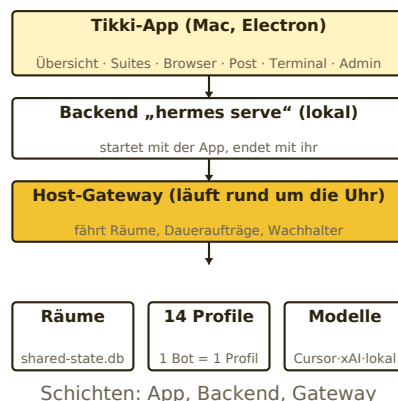
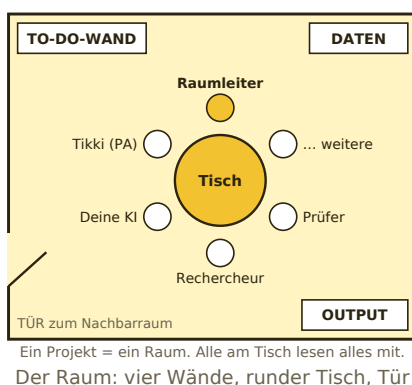
Tikki ist eine eigene Mac-App auf Basis des Open-Source-Agenten **Hermes Agent** (Nous Research, MIT). Sie macht aus einem KI-Agenten ein **Haus mit Räumen**: jedes Projekt bekommt einen Raum, in dem ein Raumleiter mit beliebig vielen spezialisierten Bots arbeitet – auch dann, wenn die App geschlossen ist. Am Eingang steht die persönliche Assistentin **Tikki**: sie empfängt, gibt das Briefing, erledigt Post und Daueraufträge und macht aus Wünschen Räume. Der Hermes-Kern ist bis auf zwei kleine, dokumentierte Stellen unverändert; alles Eigene liegt in **tikki/** und in den App-Bereichen.

2 • Die sechs Bereiche der App (linke Leiste)

Übersicht – Raumübersicht mit Tikki: Begrüßung, Tagesbriefing (erledigte Aufträge, Post, Räume, die den Menschen brauchen, WhatsApp), Karte „Daueraufträge“, Browserzeile. Sagt man „Plan mir ...“, antwortet Tikki mit RAUM: und ZIEL:, und der Raum öffnet sich. **Suites** – die Räume (Abschnitt 3). **Browser** – eingebaut, klickt Cookie-Banner selbst weg. **Post** – eigener Mail-Client für name@tikki.team. **Terminal** – Kommandozeile. **Admin** – Bots, Gedächtnis, Betrieb (Übungsläufe, Briefing-Automatik). Design „Gelbes Glas“: gelbe Glasflächen, Raumbild in Zentralperspektive, kein Hermes-Branding sichtbar.

3 • Räume, Bots, Dauerbetrieb

Ein Raum ist technisch ein **gehosteter Gruppenraum von Hermes**. In jedem Raum sitzen von Anfang an drei: der **Raumleiter** (Grok 4.7 über das Cursor-Abo, Ausweich xAI), **Tikki** (bringt Wissen über den Menschen, Post, Aufträge) und **Deine KI** (persönliches Modell, später lokal, mit Sprache). Dazu holt der Raumleiter Rollen aus dem Katalog: Rechercheur, Prüfer, Schreiber, Frontend- und Backend-Entwickler, Sicherheitsbeauftragter, Datenanalyst, Organisator, API-Fachmann, Übersetzer, Wachhalter. Alle lesen alles mit; wer nichts beizutragen hat, schweigt. Der Raumleiter führt mit festen Zeilen: STAND:, AUFGABEN: (To-do-Wand), BRAUCHE: (eine Entscheidung des Menschen), FERTIG:. **Türen** tragen eine Nachricht in einen anderen Raum, **Verschmelzen** macht aus zwei Räumen einen mit vereinter Besatzung. Räume mit TAKT: („täglich 06:00“) laufen als Dauerauftrag; der **Wachhalter** weckt alle 15 Minuten stille Räume; **Übungsläufe** lassen denselben Auftrag parallel mit anderen Ansätzen laufen, das erste fertige Ergebnis gewinnt.



- 1 **Zweig holen**
git clone -b tikki-app ...
- 2 **installieren.sh**
Kern, 14 Rollen, App, Gateway, Selbsttest
- 3 **open -a Tikki**
oder DMG aus den GitHub-Releases

Schlüssel: nur in der Datei cvcv.txt
schlüssel-einlesen.sh liest sie ein, zeigt nie Werte
Einrichten in drei Schritten

4 • Unter der Haube

Die App (Electron, React) spricht per JSON-RPC mit einem lokalen Backend **hermes serve**. Jeder Bot ist ein eigenes **Hermes-Profil** (eigene Konfiguration, eigenes Modell mit Ausweichkette, eigene Werkzeuge, eigene SOUL-Datei als Persönlichkeit); 14 Profile richtet das Skript rollen-einrichten.sh ein. Genau **ein Host-Gateway** aus dem Hauptprofil läuft rund um die Uhr und bedient alle Profile: es fährt die Räume (Speicher shared-state.db), die Daueraufträge (Hermes-Cronjobs), Takt und Wachhalter. **Gedächtnis**: Honcho als Anbieter plus das Plugin gedaechtnis (Wissensdatenbank je Mensch, RAG, Werkzeug nachschlagen). **Post**: Plugin pa mit Werkzeug post (lesen, antworten, senden; „schick weg“ heißt schicken) und briefing_sammeln. **Modelle**: über Abos (Cursor, Claude, Codex, Gemini, Grok) oder Schlüssel; ein lokales Modell über den Anbieter lokal (braucht mindestens 64k Kontext). Schlüssel stehen nur in der Datei ~/.hermes/.env, nie im Code, Chat oder in SOULs.

5 • Einrichten, aktualisieren, prüfen

Ein Befehl: ~/.hermes/hermes-agent/tikki/installieren.sh (Hermes-Kern, Tikki.app nach /Programme, Rollen, Plugins, Cronjobs, Schlüssel aus cvcv.txt, Gateway, Selbsttest ✓/△/X). Aktualisieren: git pull und dasselbe Skript. Prüfen: selbsttest.py. Für einen zweiten Rechner baut der GitHub-Workflow „Tikki App (macOS Download)“ ein DMG unter Releases (unsigned: beim ersten Start Rechtsklick → Öffnen). Von der Kommandozeile: raeume.py anlegen[senden]verlauf[verschmelzen]tuer, suite_takt.py takt|bericht, hermes pa briefing|post.

6 • Stand und offene Punkte

Gepprüft: Installation auf dem Mac (Selbsttest 7 ✓), Räume in der App gegen das laufende Gateway (anlegen, Auftrag, Türen, Verschmelzen), App-Tests 12.402 grün, Tikki-Tests 44 grün. **Noch nicht gebaut:** MR-Bot (behält Schlüssel, Limits, Abos und lokale Modelle im Blick und meldet dem Raumleiter), Sprache (lokale KI mit Stimme, xAI Ara als zweiter Kanal), 3D-Oberfläche, lokales MLX-Modell (70B/122B MoE, unzensiert) im Vergleich zur API, Login nur für eigene Domains mit PIN per Mail (erst nach der Testphase), feste API-Regeln, Raumleiter-Klone für echte Parallelität.

Quellen im Repo: tikki/HANDOVER.md (Übergabe, Abschnitte 1.0, 3, 4.2a', 4.7a, 4.8, 10), tikki/README.md, tikki/rollen/KATALOG.json, Draft-PR github.com/TikkiAL-boop/hermes-agent/pull/1.